Druid

介绍

Druid 是一个由阿里巴巴开源的数据库连接池，它在 Java 领域被广泛应用。它不仅是一个强大的数据库连接池，还提供了强大的监控和扩展功能，使其成为管理数据库连接的理想选择。

Druid 的主要特点包括：

- **性能卓越**：在许多测试中，Druid 的性能都优于其他常见的连接池（如 C3P0、DBCP 和 HikariCP）。
- **强大的监控功能**：Druid 提供了非常详细的数据库活动监控，包括 SQL 执行时间、并发数、连接泄露、慢 SQL 等，并可以通过内置的 Web 界面进行实时查看。
- **SQL 防火墙**：可以配置规则来防止恶意的 SQL 注入攻击和不安全的 SQL 操作。
- **丰富的功能**：支持数据源加密、多数据源、Statement 缓存、统计分析等。
- **易于扩展**：提供了丰富的扩展点，方便开发者根据需求进行定制。

Druid 不仅仅是连接池，它更像是一个集成解决方案，用于数据库连接的管理、监控和安全。

Druid 连接池配置参数详解

这些参数通常在 Spring Boot 的 `application.properties` 或 `application.yml` 中以 `spring.datasource.druid.` 为前缀进行配置，或者在 Java 代码中直接设置 `DruidDataSource` 对象的属性。

1. 基本连接参数 (Basic Connection Parameters)

这些是连接数据库所必需的参数。

- `spring.datasource.url`：数据库连接 URL。
  - 示例: `jdbc:mysql://localhost:3306/your_database?useUnicode=true&characterEncoding=UTF-8&serverTimezone=Asia/Shanghai`
- `spring.datasource.username`：数据库用户名。
- `spring.datasource.password`：数据库密码。
- `spring.datasource.driver-class-name`：数据库驱动类名。
  - 示例: `com.mysql.cj.jdbc.Driver` (MySQL 8+)，`com.mysql.jdbc.Driver` (MySQL 5)，`org.postgresql.Driver` (PostgreSQL)，`oracle.jdbc.OracleDriver` (Oracle)
- `spring.datasource.type`：指定数据源类型为 DruidDataSource。
  - 示例: `com.alibaba.druid.pool.DruidDataSource` (在 Spring Boot 中使用 `druid-spring-boot-starter` 时，通常会自动推断，但显式指定更稳妥)

2. 连接池大小参数 (Connection Pool Size Parameters)

这些参数控制连接池中连接的数量。

- `spring.datasource.druid.initial-size`：**初始化时建立的物理连接个数。**在连接池启动时就创建指定数量的连接。
  - 默认值: `0`

来狗安全内部培训资料

- `spring.datasource.druid.min-idle`：**最小空闲连接数。**连接池中保持的最小空闲连接数。即使没有活动连接，也会保持这么多连接。
  - 默认值: `0`
- `spring.datasource.druid.max-active`：**最大活动连接数。**连接池中允许的最大连接数。当连接数达到此值时，新的请求将等待。
  - 默认值: `8`
- `spring.datasource.druid.max-wait`：**获取连接时最大等待时间**，单位毫秒。当连接池中没有可用连接时，客户端等待获取连接的最长时间。超过此时间将抛出异常。
  - 默认值: `-1` (永不超时)

3. 连接检测与维护参数 (Connection Validation and Maintenance Parameters)

这些参数用于确保连接的可用性和健康状况。

- `spring.datasource.druid.time-between-eviction-runs-millis`：**检测需要关闭的空闲连接的间隔时间**，单位毫秒。Druid 会定期运行一个线程来检测并关闭长时间空闲的连接。
  - 默认值: `60000` (1分钟)
- `spring.datasource.druid.min-evictable-idle-time-millis`：**连接在连接池中最小的生存时间**，单位毫秒。如果一个连接空闲时间超过此值，并且 `time-between-eviction-runs-millis` 线程检测到它，则该连接会被关闭。
  - 默认值: `1800000` (30分钟)
- `spring.datasource.druid.validation-query`：**用于检测连接是否有效的 SQL 查询语句。**Druid 在获取连接或归还连接时，会执行此 SQL 来验证连接是否仍然可用。
  - 示例: `SELECT 1` (MySQL/PostgreSQL)，`SELECT 1 FROM DUAL` (Oracle)
  - 默认值: `null`
- `spring.datasource.druid.test-while-idle`：**申请连接的时候是否检测连接有效。**建议设置为 `true`。
  - 默认值: `true`
- `spring.datasource.druid.test-on-borrow`：**归还连接的时候是否检测连接有效。**通常设置为 `false`，因为 `test-while-idle` 已经足够。如果设置为 `true`，会增加每次获取连接的开销。
  - 默认值: `false`
- `spring.datasource.druid.test-on-return`：**归还连接的时候是否检测连接有效。**同 `test-on-borrow`，通常设置为 `false`。
  - 默认值: `false`
- `spring.datasource.druid.pool-prepared-statements`：**是否缓存 PreparedStatement。**开启可以提高性能，特别是对于频繁执行相同 SQL 的场景。
  - 默认值: `false`
- `spring.datasource.druid.max-pool-prepared-statement-per-connection-size`：**每个连接最多缓存多少个 PreparedStatement。**当 `pool-prepared-statements` 为 `true` 时有效。
  - 默认值: `10`

柔狗安全内部培训资料

4. 监控与安全参数 (Monitoring and Security Parameters)

这些参数用于开启 Druid 的强大监控和安全功能。

- `spring.datasource.druid.filters`: **过滤器链**。用于开启 Druid 的各种功能。多个过滤器之间用逗号分隔。
  - `stat`: **统计监控**。必须开启才能在 Druid 监控页面看到数据。
  - `wall`: **SQL 防火墙**。可以防止 SQL 注入、DDL 限制等。
  - `log4j` / `log4j2` / `slf4j` / `commons-logging`: **SQL 日志记录**。根据你项目使用的日志框架选择。
  - `encoding`: 字符集编码。
  - `config`: 用于通过配置文件加载额外配置。
  - 示例: `stat,wall,log4j2` (最常用组合)
- `spring.datasource.druid.connection-properties`: **额外的 JDBC 连接属性**。可以用来传递数据库驱动特定的属性，或 Druid 统计相关的属性。
  - `druid.stat.mergeSql=true`: 将相同的 SQL 语句合并统计，方便查看。
  - `druid.stat.slowSqlMillis=5000`: 定义慢 SQL 的阈值，超过此时间的 SQL 会被标记为慢 SQL。
  - 示例: `druid.stat.mergeSql=true;druid.stat.slowSqlMillis=5000`

5. 监控页面相关参数 (Monitoring Web UI Parameters)

这些参数用于配置 Druid 监控页面的访问。

- `spring.datasource.druid.stat-view-servlet.enabled`: **是否启用 Druid 监控页面**。
  - 默认值: `false` (在 `druid-spring-boot-starter` 中，如果检测到 Actuator 依赖，可能会自动设置为 `true`)
- `spring.datasource.druid.stat-view-servlet.url-pattern`: **监控页面的访问路径**。
  - 示例: `/druid/*`
- `spring.datasource.druid.stat-view-servlet.login-username`: **监控页面登录用户名**。
- `spring.datasource.druid.stat-view-servlet.login-password`: **监控页面登录密码**。
- `spring.datasource.druid.stat-view-servlet.allow`: **允许访问监控页面的 IP 白名单**。多个 IP 用逗号分隔。空或不配置表示允许所有 IP。
  - 示例: `127.0.0.1,192.168.1.0/24`
- `spring.datasource.druid.stat-view-servlet.deny`: **拒绝访问监控页面的 IP 黑名单**。优先级高于 `allow`。
- `spring.datasource.druid.stat-view-servlet.reset-enable`: **是否允许在监控页面重置数据**。设置为 `false` 可以防止误操作。
  - 默认值: `true`

durid的配置方法

一种就是通过项目配置文件配置，例如：application.properties，还有一种是通过创建druidStatViewServlet进行配置

柔狗安全内部培训资料

例如：

```
@Configuration
public class DruidConfig {

    @Bean
    public ServletRegistrationBean<StatViewServlet> druidStatViewServlet() {
        ServletRegistrationBean<StatViewServlet> reg = new
ServletRegistrationBean<>(
            new StatViewServlet(), "/druid/*");

        reg.addInitParameter("loginUsername", "admin");
        reg.addInitParameter("loginPassword", "admin123");

        reg.addInitParameter("resetEnable", "false");
        return reg;
    }
}
```

关于durid的审计点

1、是否存在未授权访问的情况

2、是否存在硬编码可以登入

1、是否存在未授权访问的情况

详细说下，未授权访问这个点网上的界定很模糊，正常来说应该是在没有登入的情况下可以直接进入durid管理后台，才叫做未授权访问，但是很多时候，只要在前端能够访问到durid登入接口好像就已经可以叫做未授权访问了，因为从开发安全角度来说这种第三方的管理系统就应该是在管理员后台才能访问的，而且正常来说还需要配置白名单访问之类的，所以这种未授权访问在生产环境中确实是可以可以整改的点，也就是水报告，但是我这里主要是说下配置不当导致未授权登入管理后台

如果在配置的时候账号密码留空，那么在访问/durid/的时候就会直接登入

application.properties：

```
# 开启Druid监控
spring.datasource.druid.stat-view-servlet.enabled=true
spring.datasource.druid.stat-view-servlet.url-pattern=/druid/*

#spring.datasource.druid.stat-view-servlet.login-username=admin
#spring.datasource.druid.stat-view-servlet.login-password=admin123
```

DruidConfig：

```
@Configuration
public class DruidConfig {

    @Bean
    public ServletRegistrationBean<StatViewServlet> druidStatViewServlet() {
        ServletRegistrationBean<StatViewServlet> reg = new
ServletRegistrationBean<>(
            new StatViewServlet(), "/druid/*");
```

柔和安全内部培训资料

```
        reg.addInitParameter("loginUsername", "caigo");
        reg.addInitParameter("loginPassword", "caigo");

        reg.addInitParameter("resetEnable", "false");
        return reg;
    }
}
```

在早版本可以使用

```
# 禁用Druid监控页面的登录验证
spring.datasource.druid.stat-view-servlet.login-enable=false
```

来禁用登入验证

2、是否存在硬编码可以登入

关注点也是两个，一个是配置文件：application.properties，还有一个就是是否有自己注册druidStatViewServlet，关于硬编码主要是关注druidStatViewServlet，因为这个是内嵌在代码里面的，而application.properties是可以外部修改的，很多运维在搭建的时候不一定会关注到druidStatViewServlet

Spring Boot Actuator

介绍

Spring Boot Actuator模块提供了生产级别的功能，比如健康检查，审计，指标收集，HTTP跟踪等，帮助我们监控和管理SpringBoot应用。

Spring Boot Actuator端点通过 JMX 和HTTP 公开暴露给外界访问，大多数时候我们使用基于HTTP的Actuator端点，因为它们很容易通过浏览器、CURL命令、shell脚本等方式访问。

Actuator各端点的作用

柔狗安全内部培训资料

请求方法	端点	描述
GET	/actuator	查看有哪些 Actuator端点是开放的。
GET	/actuator/auditevent	auditevents端点提供有关应用程序审计事件的信息。
GET	/actuator/beans	beans端点提供有关应用程序 bean 的信息。
GET	/actuator/conditions	conditions端点提供有关配置和自动配置类条件评估的信息。
GET	/actuator/configprops	configprops端点提供有关应用程序@ConfigurationPropertiesbean的信息。
GET	/actuator/env	查看全部环境属性，可以看到 SpringBoot 载入哪些 properties，以及 properties 的值（会自动用*替换 key、password、secret 等关键字的 properties 的值）。
GET	/actuator/flyway	flyway端点提供有关 Flyway 执行的数据库迁移的信息。
GET	/actuator/health	端点提供有关应用程序运行状况的health详细信息。
GET	/actuator/heapdump	heapdump端点提供来自应用程序 JVM 的堆转储。(通过分析查看/env端点被*号替换到数据的具体值。)
GET	/actuator/httptrace	httptrace端点提供有关 HTTP 请求-响应交换的信息。（包括用户HTTP请求的Cookie数据，会造成Cookie泄露等）。
GET	/actuator/info	info端点提供有关应用程序的一般信息。
GET	/actuator/integrationgraph	integrationgraph端点公开了一个包含所有 Spring Integration 组件的图。
GET	/actuator/liquibase	liquibase端点提供有关 Liquibase 应用的数据库更改集的信息。
GET	/actuator/logfile	logfile端点提供对应用程序日志文件内容的访问。
GET	/actuator/loggers	loggers端点提供对应用程序记录器及其级别配置的访问。
GET	/actuator/mappings	mappings端点提供有关应用程序请求映射的信息。
GET	/actuator/metrics	metrics端点提供对应用程序指标的访问。
GET	/actuator/prometheus	端点以prometheusPrometheus 服务器抓取所需的格式提供 Spring Boot 应用程序的指标。
GET	/actuator/quartz	quartz端点提供有关由 Quartz 调度程序管理的作业和触发器的信息。
GET	/actuator/scheduledtasks	scheduledtasks端点提供有关应用程序计划任务的信息。
GET	/actuator/sessions	sessions端点提供有关由 Spring Session 管理的应用程序 HTTP 会话的信息。
GET	/actuator/startup	startup端点提供有关应用程序启动顺序的信息。
POST	/actuator/shutdown	shutdown端点用于关闭应用程序。

某猫安全内部培训资料

审计关注点

项目如果引用了actuator，关注开放了哪些端点，根据端点功能进行利用

Swagger 介绍

Swagger 是一套用于 **API 设计、文档生成、测试和可视化** 的开源工具集，旨在解决 API 开发过程中文档维护困难、接口测试繁琐等问题。它允许开发者通过代码注解或配置的方式，自动生成交互式 API 文档，同时支持在线调试接口，极大提升了前后端协作效率。

核心功能

1. **自动生成 API 文档**
   通过在代码中添加注解（如 `@Api`、`@ApiOperation` 等），Swagger 可自动解析接口信息（包括请求方法、参数、返回值、数据类型等），生成标准化的文档，避免手动编写文档的繁琐和不一致。

2. **交互式接口测试**
   生成的文档提供 **在线调试界面**（类似 Postman），开发者可直接在页面上输入参数、发送请求并查看响应结果，无需额外工具即可测试接口。
3. **支持多种语言和框架**
   适用于 Java（Spring Boot、Spring MVC）、Python（Flask、Django）、Node.js 等主流语言和框架，兼容性强。
4. **API 标准化**
   基于 OpenAPI 规范（原 Swagger 规范），定义 API 的格式和元数据，确保接口文档的一致性和可扩展性，便于团队协作和自动化工具集成（如 API 网关、测试工具）。

常用工具组件

Swagger 生态包含多个工具，核心组件如下：

- **Swagger Editor**：可视化 API 设计工具，支持通过 YAML/JSON 编写 OpenAPI 规范，实时预览文档。
- **Swagger UI**：将 OpenAPI 规范转换为交互式 HTML 文档的工具，提供接口列表、参数说明和调试功能（Spring Boot 中常用此组件）。
- **Swagger Codegen**：根据 OpenAPI 规范自动生成客户端 SDK（如 Java、Python 客户端）和服务端代码骨架，简化开发流程。
- SpringDoc/ SpringFox
  - **SpringFox**：早期用于 Spring 项目集成 Swagger 的工具（基于 Swagger 2.x）。
- **SpringDoc**：新一代工具，支持 OpenAPI 3.0 规范，更适配 Spring Boot 2.x/3.x，推荐使用。

柔狗安全内部培训资料